

Prüfungsfragen - Prüfung 2 (22.05)

25%

Kompetenzen: D1G, E1F, F1E, G1G

D1G → 3P



Lernziel:

Ich kann die Funktion von Zugriffsberechtigungen in einer NoSQL Datenbank erläutern (Benutzer, Rollen, Zugriffsrechte)

Frage:

Ein Entwickler erstellt in Redis den Benutzer readonly mit folgendem Befehl:

ACL SETUSER readonly on >pass123 ~* +GET

- a) Erklären Sie, was die einzelnen Bestandteile des Befehls bedeuten. (1P)
- b) Welche Aktionen darf dieser Benutzer ausführen und welche nicht? (1P)
- c) Ordnen Sie die Begriffe korrekt zu/ verbinde es: (1P)

Benutzer	nur GET
Rolle	<u>readonly</u>
Zugriffsrechte	Leser (<u>readonly</u>)

Lösung:

- a)
 - **ACL SETUSER** → Ändern oder Erstellen eines Users
 - **readonly** → Benutzername
 - **on** → Benutzer aktiv
 - **>pass123** → Passwort
 - **~*** → Zugriff auf alle Keys
 - **+GET** → nur Lesen erlaubt

- b)
 - darf: Daten lesen (GET)
 - darf nicht: schreiben (SET), löschen (DEL)

- c)
 - Benutzer → readonly
 - Rolle → Leser (readonly)
 - Zugriffsrechte → nur GET

E1G → 3P



Lernziel:

Ich kann Konzepte für ein Backup einer NoSQL Datenbank erläutern (z. B. on-demand snapshots, continuous cloud backups, legacy backups)

Frage:

Beschreiben Sie zwei konkrete Backup-Mechanismen von Redis und erklären Sie jeweils wie diese funktionieren.

Gehen Sie dabei auf den Unterschied zwischen den beiden Methoden ein.

Lösung:

Kriterium	Punkte
RDB (Snapshot) korrekt beschrieben (Zeitpunkt-Speicherung)	1
AOF korrekt beschrieben (jede Operation wird geloggt)	1
Klarer Unterschied erklärt (z. B. Performance vs. Genauigkeit)	1

Keine Punkte für:

- nur aufzählen ohne Erklärung
- allgemeine Backup-Antwort ohne Redis-Bezug

Musterlösung:

1. RDB (Snapshot):

- a. Dabei wird der komplette Datenstand zu bestimmten Zeitpunkten gespeichert.
- b. Das ist schnell und platzsparend, aber es können Daten zwischen zwei Snapshots verloren gehen.

2. AOF (Append Only File):

- a. Hier wird jede Schreiboperation (z. B. SET) in einer Datei gespeichert.
- b. Beim Neustart kann Redis alle Befehle wieder abspielen.

Unterschied:

- RDB ist schneller und effizienter, aber bei einem Absturz sind die geretteten Daten weniger aktuell.
- AOF ist aktueller (fast kein Datenverlust), aber langsamer und benötigt mehr Speicher.
- Da Redis im RAM arbeitet, sind solche Backups wichtig

F1G → 3P**Lernziel:**

Ich kann das Prinzip der Skalierung und die unterschiedlichen Replikationsarten für eine NoSQL Datenbank erläutern (z. B. Multimaster, primary and replica, Aktiv-Passiv und horizontale Skalierung)

Frage:

Ein Start-up erwartet stark wachsende Nutzerzahlen und muss seine Redis-Datenbank skalieren

a) Was bedeutet «horizontale Skalierung»? Kreuze an. (1 Punkt)

- ☐ a. Den vorhandenen Server mit mehr RAM ausstatten
- ☐ b. Mehr Server/Knoten zum System hinzufügen
- ☐ c. Die Datenbank auf eine schnellere SSD migrieren
- ☐ d. Die Anzahl der Benutzer reduzieren

b) Erkläre den Unterschied zwischen «Sharding» und «Replikation» in Redis. (1 Punkt)

c) Was versteht man unter «Hochverfügbarkeit» und wie wird sie in Redis erreicht? (1 Punkt)

Lösung**Keine Punkte für:**

- Nur Begriffe nennen ohne Erklärung (z.B. nur «mehr Server» ohne zu erklären was das bedeutet)
- Sharding und Replikation verwechseln oder gleichsetzen
- Allgemeine Antwort ohne Redis-Bezug
- Hochverfügbarkeit nur mit «das System läuft immer» erklären, ohne zu beschreiben wie Redis das erreicht

Musterlösung:

a) Was bedeutet horizontale Skalierung? (1 Punkt)

- ☒ B Mehr Server/Knoten zum System hinzufügen

b) Sharding vs. Replikation (1 Punkt)

Sharding: Die Daten werden auf mehrere Knoten aufgeteilt (z.B. Shard 1 hat Keys A–M, Shard 2 hat Keys N–Z). Ziel ist Lastverteilung.

Replikation: Dieselben Daten werden auf mehrere Knoten kopiert und synchron gehalten. Ziel ist Ausfallsicherheit und Leseleistung.

c) Hochverfügbarkeit in Redis (1 Punkt)

Hochverfügbarkeit bedeutet, dass das System auch bei Ausfällen einzelner Knoten weiter verfügbar bleibt. Redis erreicht dies durch Replikation (Primary + Replicas) und automatisches Failover: fällt der Primary aus, übernimmt eine Replica automatisch.

G1G → 3P



Lernziel:

Ich kann das Prinzip des Zugriffes bei einer NoSQL Datenbank erläutern (z. B. Queries, Projections)

Frage:

1. Was ist Redis primär? Zeichne ein kurzes Beispiel dazu mit 3 Einträgen. (1P)
2. Wie greift man in Redis auf einen Wert zu? (1P)
3. Was versteht man in NoSQL unter einer Projection? (1P)

Lösung:

1. Redis ist ein In-Memory Key-Value Store. Das bedeutet: alle Daten liegen im RAM, nicht auf der Festplatte. Dadurch sind Zugriffe extrem schnell (Mikrosekunden-Bereich).
 - Man kann sich Redis wie ein riesiges Wörterbuch vorstellen, jeder Eintrag hat einen eindeutigen Schlüssel (Key) und einen Wert (Value).

```
Key      → Value
"username" → "alice99"
"score"   → "1500"
"online"  → "true"
```

2. Über den direkten Key-Lookup. Man muss den Key kennen, es gibt kein "Suchen" wie in SQL.

```
SET username "alice99"    # Wert speichern
GET username              # → "alice99"
```

- Bei einem Hash (mehrere Felder unter einem Key):

```
HSET user:1001 name "Alice" age "23" city "Bern"
HGET user:1001 name      # → "Alice"
HGETALL user:1001        # → alle Felder
```

- Der Zugriff ist $O(1)$ – egal wie viele Einträge es gibt, der Zugriff dauert gleich lang. Das ist der fundamentale Unterschied zu SQL, wo ein WHERE-Filter oft einen Tabellenscan auslöst.

3. Projection bedeutet: man holt nur bestimmte Felder eines Eintrags, nicht alles. Das reduziert den Datenverkehr und ist effizienter.

In SQL kennt man das so:

```
sql
SELECT name, city FROM users WHERE id = 1001;
-- nur name und city, nicht age, email, etc.
```

In Redis macht man das mit `HMGET` bei Hashes:

```
redis
HSET user:1001 name "Alice" age "23" city "Bern" email "alice@mail.ch"

HMGET user:1001 name city
# → "Alice"
# → "Bern"
# age und email werden nicht übertragen
```

Befehl	Felder	Effizienz
<code>HGETALL user:1001</code>	alle Felder	mehr Daten, kein Filter
<code>HMGET user:1001 name city</code>	nur name + city	Projection, weniger Overhead